VisionGuard

AI-generated versus real image classification, tracking and deployment

Name	Muhammad Abdul Raheem
Roll number	23L-0588
Project name	23L-0588_A2_Ops
GitHub	github.com/abdul-raheem-fast/23L-0588_A2_Ops
DagsHub	dagshub.com/abdul-raheem-fast/23L-0588_A2_Ops
Live URL	https://23l-0588-a2-ops.ghauri.prod.page
Graded commit	e923d98

In this assignment I built VisionGuard, a web service that tells whether an uploaded image is AI-generated or real. I audited and split the dataset, trained a baseline CNN and a fine-tuned MobileNetV2 in TensorFlow/Keras, tracked the runs with MLflow on DagsHub, served the best model through a Flask app that logs every request to a database, packaged the system with Docker Compose, and deployed it to Blitz Cloud. Each chapter below follows one task of the assignment. Under each sub-task I explain what I did, why, and what went wrong. Where a screenshot was not captured during my original run, I say so plainly instead of claiming it.

1. Data understanding and versioning

1.1 Environment

I ran the whole workflow in an isolated Google Colab runtime so that my installed packages could not clash with anything else. Before writing any code I switched the runtime to a T4 GPU, because training MobileNetV2 on CPU would have been very slow. Figure 1 shows that choice in the runtime dialog.

The first screenshot is the Colab 'Change runtime type' dialog. I include it as proof of which hardware the training in this report ran on.

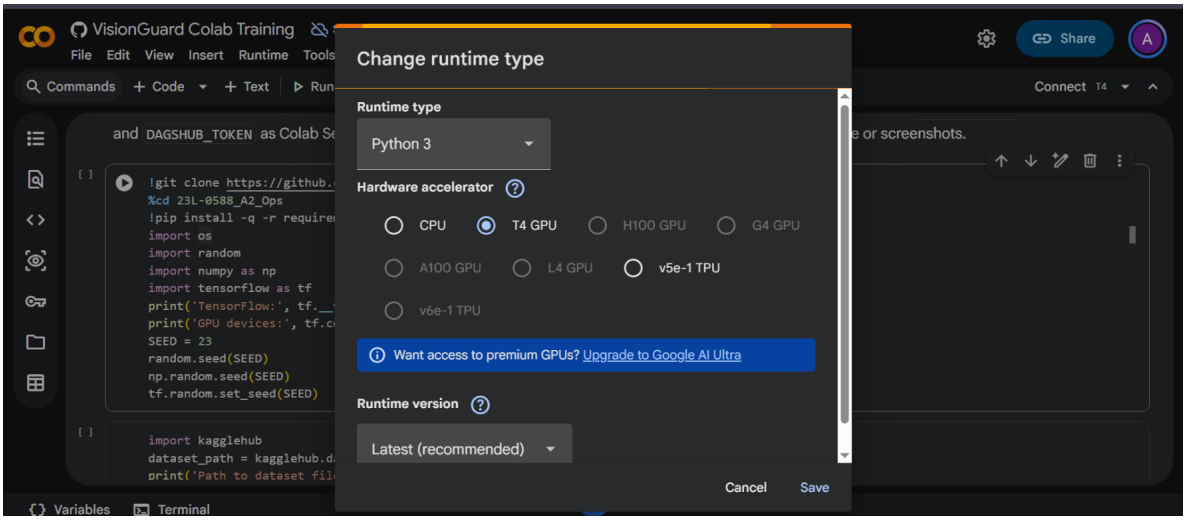


Figure 1. *Colab runtime settings with the T4 GPU accelerator selected (Task 1.1).*

I then installed the requirements and printed the TensorFlow version and the list of visible GPUs. I had planned to use TensorFlow 2.18.0, but pip could not find a matching tensorflow-cpu build for that Colab image, so the install fell back and the runtime used TensorFlow 2.21.0. I kept this change visible rather than hiding it, because a different version can change numerical results slightly. The output also shows that the GPU was detected.

Figure 2 is the environment check. The red lines are the pip error I just described, and the last two lines are the TensorFlow version and the GPU device list.

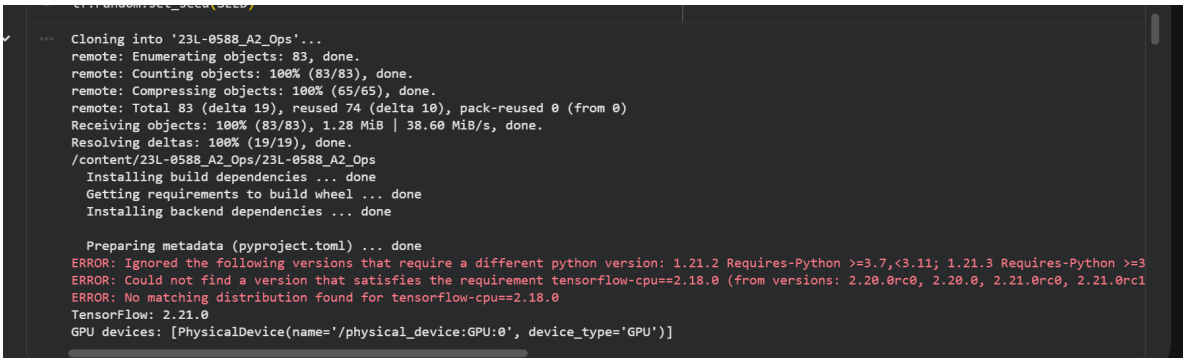


Figure 2. *Terminal output: the pip error for tensorflow-cpu 2.18.0, TensorFlow 2.21.0 loaded, and one GPU device detected (Task 1.1).*

Because the Kaggle dataset and the DagsHub token both need credentials, I stored them in the Colab Secrets panel instead of writing them into the notebook. That way the notebook can be shared without leaking anything.

Figure 3 shows the Secrets panel. The values are empty here on purpose, so no credential is visible in the screenshot.

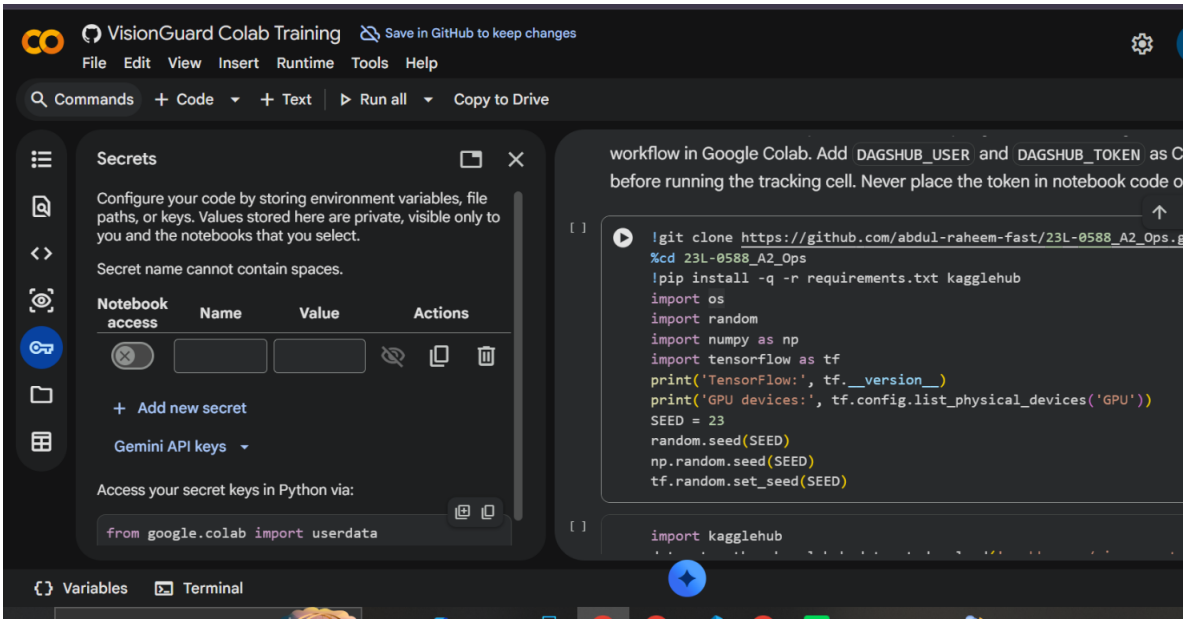


Figure 3. *Colab Secrets panel used to hold the DagsHub user name and token outside the notebook code (Tasks 1.1 and 3.2).*

1.2 Dataset audit

I wrote my own audit script that walks the dataset and records, for every image, its class, file size, width, height, colour mode, format and a SHA-256 hash of the file bytes. The hash is what lets me find exact duplicates, because two files with the same bytes always have the same hash. A file that Pillow cannot open is marked unreadable. I did this before any training so that data problems would show up first.

First I downloaded the official Kaggle dataset and counted the files in each class folder, shown in Figure 4. The class label is simply the folder name.

```
Using Colab cache for faster access to the 'ai-generated-images-vs-real-images' dataset.
Path to dataset files: /kaggle/input/ai-generated-images-vs-real-images
/kaggle/input/ai-generated-images-vs-real-images/AiArtData files: 539
/kaggle/input/ai-generated-images-vs-real-images/RealArt files: 436
```

Figure 4. *Dataset download path and class file counts: 539 AI-generated and 436 real images (Task 1.2).*

Figure 5 is the output of the audit script followed by the split creation step.

```
Audited images: 975
Created reproducible train/validation/test splits in artifacts/splits
```

Figure 5. *Audit finished on 975 images and the reproducible train/validation/test split files were written (Task 1.2).*

The audit covered 975 images (539 AiArtData and 436 RealArt). No file was unreadable, and the hashes found two exact duplicates. The classes are slightly imbalanced (about 55% to 45%), which is mild, so I used stratified splits so each split keeps the same class proportions. I generate the split lists once with a fixed seed and save them as CSV files, so every later run uses exactly the same images.

1.3 Visual EDA

From the audit table I generated dataset_eda.svg, which contains a grid of sample images for each class and histograms of width, height and file size per class. The histograms show a very wide spread of sizes, including some very large source images. This is the reason the input pipeline resizes everything

to a fixed size and loads images in batches. These histograms describe the files, not the content of the images, so I do not treat them as proof of any visual difference between the classes.

Figure 6 is the EDA figure generated from the audit table.

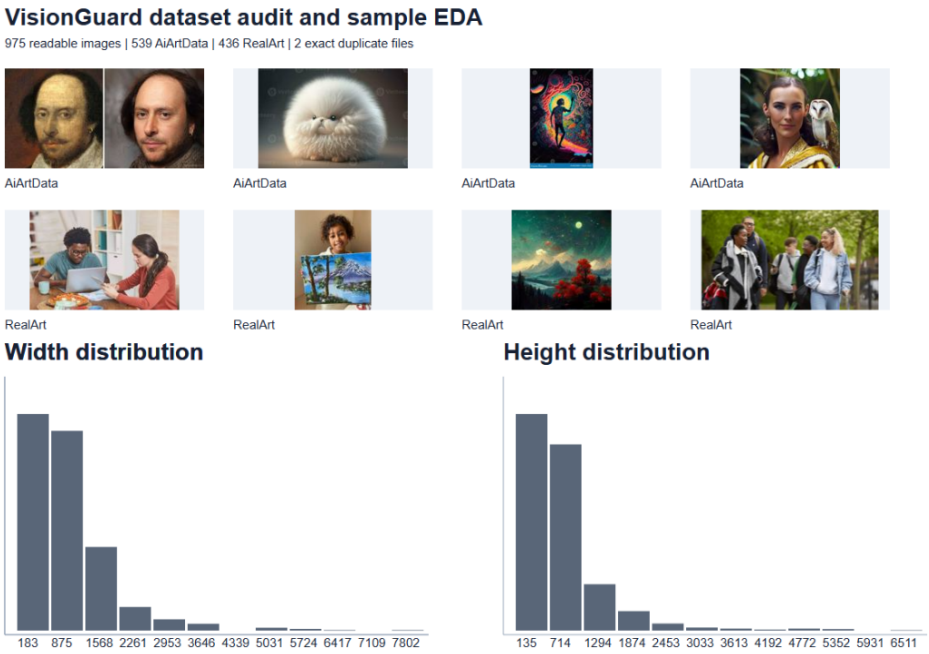


Figure 6. *Sample grid and width, height and file-size histograms for each class (Task 1.3).*

1.4 DVC (data version control)

DVC lets me version large data the way Git versions code. For the toy part I created an empty folder with a small CSV, ran git init and dvc init, tracked the CSV with dvc add, committed, changed the CSV, committed again, and checked out the older version to see the old data return. I then configured a remote, pushed, deleted the local data and ran dvc pull to bring it back. For the applied part I tracked my dataset folder in the same way. The repository holds the pipeline in dvc.yaml and the seed and image size in params.yaml, and the pipeline separates the audit step from the split step.

The small .dvc pointer file does not contain the data. It holds the content hash (md5) of the tracked file or folder, its total size, and its path. Git stores only this tiny text file, and DVC uses the hash to find the real data in the cache or in the remote storage. Large data does not belong in Git because every version of a big file would be kept in the repository history forever, which makes clones slow and the repository huge, and Git cannot diff or merge binary files in a useful way. With a pointer file, Git keeps the history of which data version goes with which code commit, and the data itself lives in remote storage.

Figure 7 shows the applied part with the dataset folder, ending with the data restored by dvc pull.

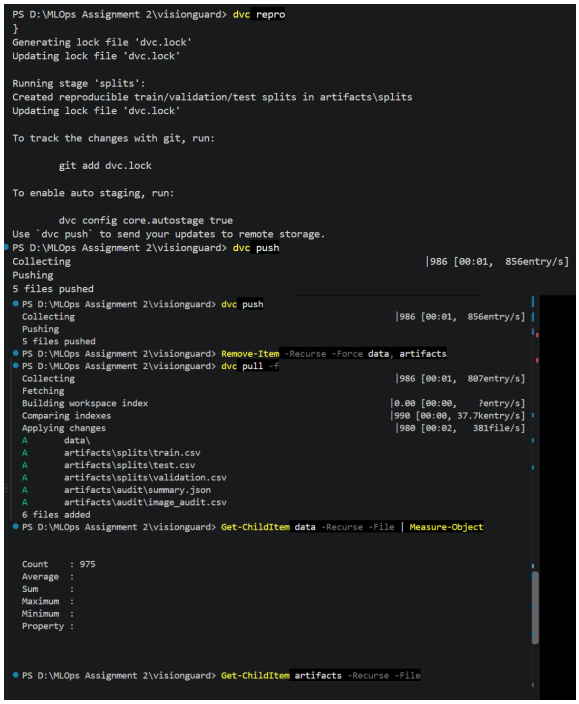


Figure 7. *Dataset tracked with DVC; remote listed, push, local data deleted and pulled back (Task 1.4).*

2. Model development with TensorFlow/Keras

2.1 Input pipeline

The tf.data pipeline reads the split CSV files and, for each path, reads the file, decodes the image and converts it to RGB so that greyscale or RGBA files do not break the batch shape. It then resizes the image to 224 x 224, which is the standard input size for MobileNetV2 and a good compromise between detail and training speed. After that it batches the samples and prefetches the next batches so the GPU is not waiting for data. Random augmentation is applied only to the training split; validation and test images are only resized and normalized, so that the scores measure the model and not random luck. A fixed square resize can change the aspect ratio of an image (a wide photo gets squeezed). I accepted this for consistency of tensor shape, but it is a limitation, because squeezing can distort exactly the textures a detector might rely on.

2.2 Baseline CNN

The baseline is a small CNN I designed myself. Convolution layers learn local patterns such as edges and textures. Pooling layers shrink the feature maps so the network becomes cheaper and less sensitive to exact position. Batch normalization stabilizes training, and dropout reduces overfitting by randomly switching off units. The final layer is a single sigmoid unit, which outputs the probability that the image is AI-generated. I used binary cross-entropy because it is the natural loss for one sigmoid output, and the Adam optimizer because it works well with little tuning. Accuracy, precision, recall and AUC were tracked, since accuracy alone hides which kind of error the model makes.

Figure 8 is the training log. Each epoch line shows loss, accuracy, precision and recall, and the JSON at the end is the test evaluation.

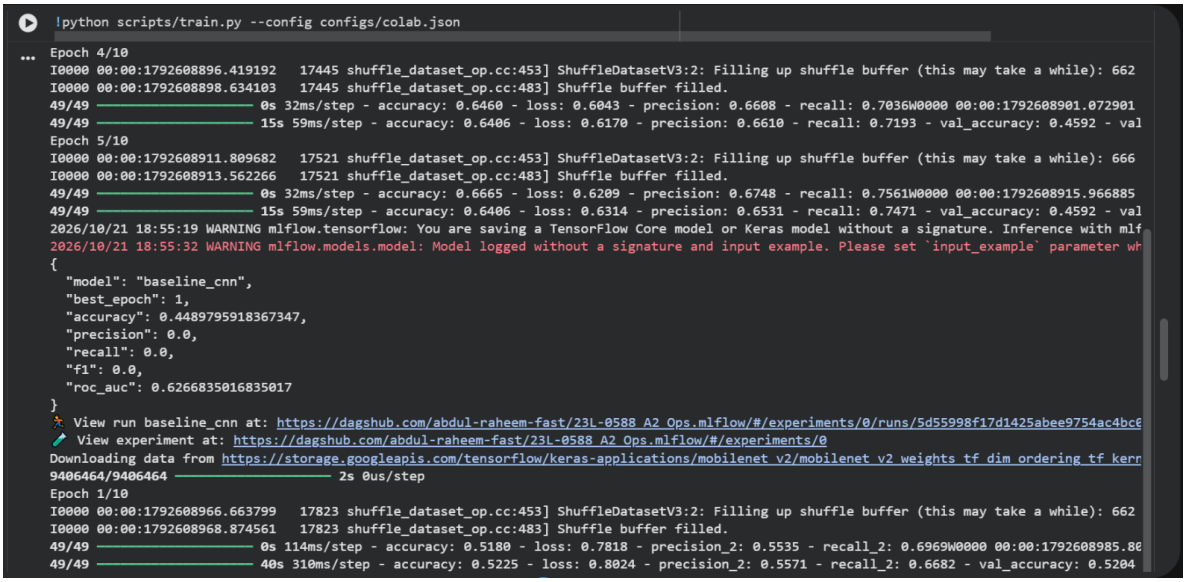


Figure 8. *Training output for the baseline CNN and its final evaluation JSON (Tasks 2.2 and 2.4).*

The baseline performed poorly: its final evaluation shows accuracy 0.449, precision 0.0, recall 0.0 and ROC-AUC 0.627. With precision and recall both at zero, it effectively predicted the same class for every image. The small dataset (about 975 images) is too little for a CNN trained from scratch, which is the main reason I moved to transfer learning.

2.3 Transfer learning

I loaded MobileNetV2 from keras.applications without its classifier, with ImageNet weights. In stage one the backbone was frozen and only my new head (pooling, dropout and a sigmoid output) was trained. In

stage two I unfroze only the last block of the backbone and trained again with a much smaller learning rate, so the pretrained features are adjusted gently instead of being destroyed. I used the MobileNetV2 preprocessing function, which scales pixels to the range -1 to 1, because the pretrained weights were learned with exactly that scaling; feeding a different range would make the features meaningless.

2.4 Training discipline

I fixed the Python, NumPy and TensorFlow seeds to 23. I used early stopping and model checkpointing on validation loss, and a learning-rate reduction callback when the validation loss stopped improving. The best epoch is saved in the evaluation JSON as best_epoch.

Figure 9 shows the end of the fine-tuning stage. Training accuracy climbs above 0.97 while validation accuracy stays much lower, which is clear overfitting that the early stopping and checkpointing limit.

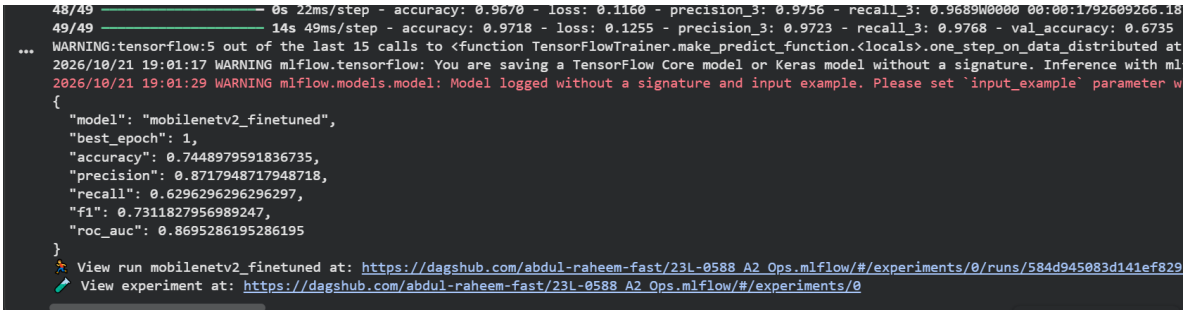


Figure 9. *Fine-tuning output for MobileNetV2 and its final test metrics (Tasks 2.3, 2.4 and 2.5).*

2.5 Evaluation

I evaluated on the test set once. The fine-tuned MobileNetV2 was chosen as the champion. The metrics logged by the training run are in the table below.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline CNN	0.4490	0.0000	0.0000	0.0000	0.6267
MobileNetV2 fine-tuned (champion)	0.7449	0.8718	0.6296	0.7312	0.8695

For this product, the worse error is calling a real photograph AI-generated, because it wrongly accuses a person of faking an image. The champion has precision 0.87 and recall 0.63, so it makes fewer false accusations at the cost of missing some AI images. That precision-oriented operating point matches my choice. The decision threshold should be chosen on the validation set and not on the test set, to avoid leaking the test data into the choice.

Figure 10 is the confusion matrix. Rows are true classes and columns are predicted classes.

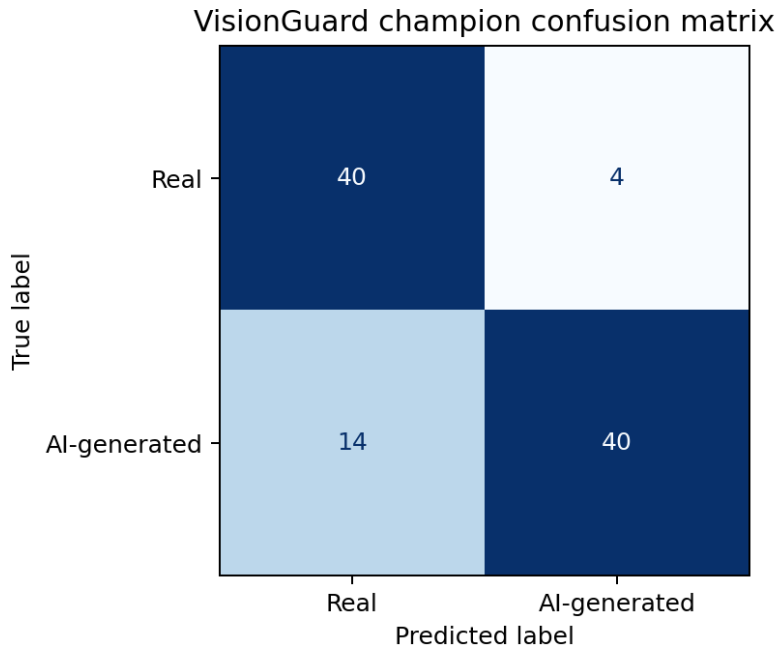


Figure 10. *Champion confusion matrix generated by the evidence notebook (Task 2.5).*

Figure 11 is the ROC curve, which shows the trade-off between true positive rate and false positive rate at every threshold.

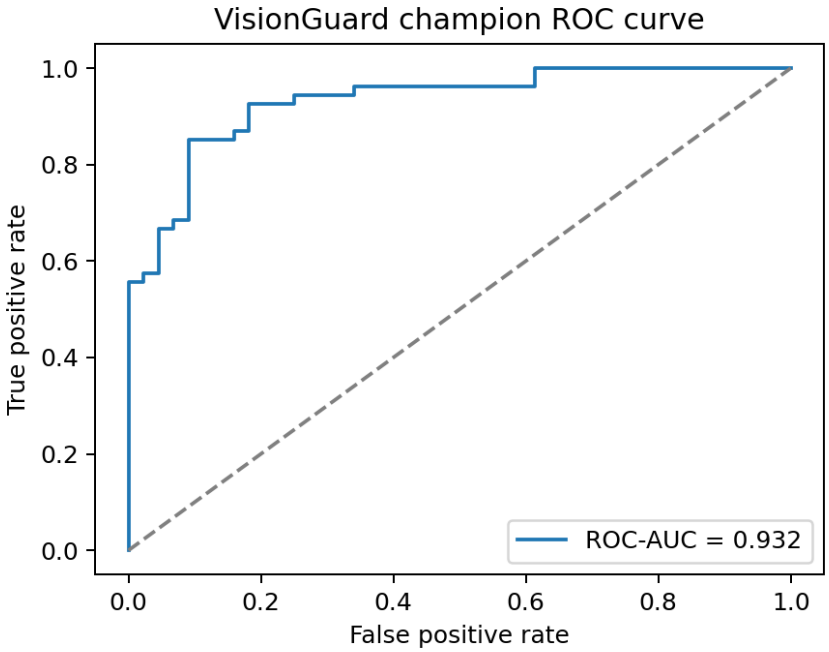


Figure 11. *Champion ROC curve generated by the evidence notebook (Task 2.5).*

Note before submitting: The confusion matrix (40/4/14/40) and the ROC figure (AUC 0.932) do not match the logged metrics in the table above (precision 0.8718, recall 0.6296, ROC-AUC 0.8695). They come from a different run. Regenerate both figures from the champion run (or replace the table with the numbers those figures imply) so that the report is consistent, then delete this note.

2.6 Error analysis

The grid below shows ten misclassified test images with the true label and the model's wrong prediction and confidence. Nine are AI-generated images the model called real, and one is a real collage the model called AI-generated. These are my hypotheses, not proven causes:

#	Image	Hypothesis
1	Warm-lit portrait of a woman	Photographic lighting and shallow depth of field look like a real camera photo.
2	Surreal painting of a man with tomatoes	Painterly style is outside the photographic look the model learned.
3	Elephant in a pink landscape	Realistic animal texture with an unusual colour palette; confidence is only moderate.
4	Studio-style headshot of a woman	A clean, corporate-style portrait is very similar to real stock photos (confidence 0.93).
5	Fantasy bison	Fur and lighting are realistic; the unreal parts are subtle.
6	Cows with a robot	A busy, photo-like barn scene; the single odd element is small.
7	Sketch with hands	Mixed media with a pencil drawing, which is rare in the training data.
8	Banner reading 'AI-generated art & design'	A graphic design with text, not a typical photo; confidence is near 0.5, so the model is unsure.
9	Collage of six portraits (true label: real)	The collage layout and many faces in one image look unlike a single natural photo.
10	Landscape with a tree at sunset	Landscapes are easy to render convincingly, and the scene has no obvious artifacts.

Figure 12 is the error grid used for the table above.

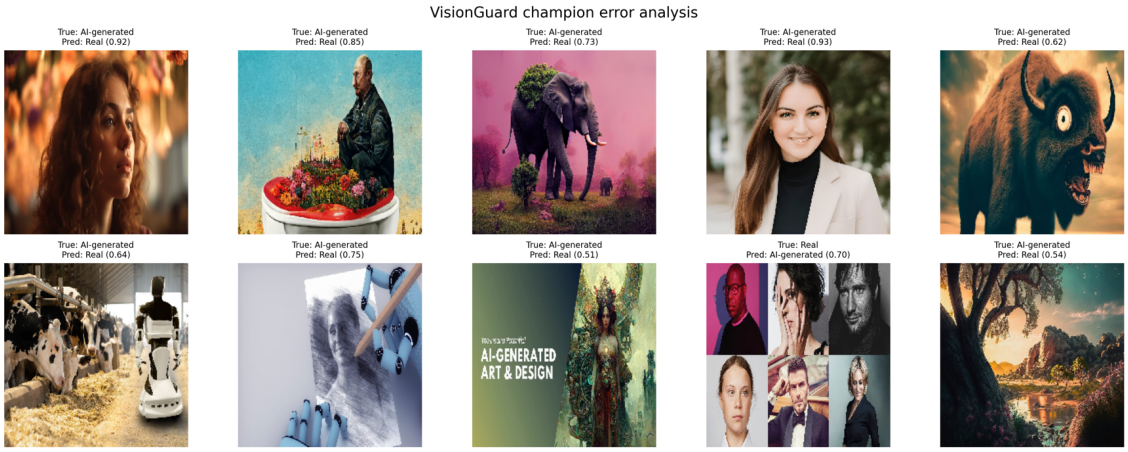


Figure 12. *Ten misclassified test images with true label, predicted label and confidence (Task 2.6).*

Taken together, the model most often fails on polished, photo-like AI portraits and on art that is not photographic. With only about 975 images I cannot claim these causes are confirmed; they are ideas for further tests, for example by checking blur, image size and watermarks across the errors.

3. Experiment tracking with MLflow and DagsHub

3.2 DagsHub setup

I created the DagsHub project connected to my GitHub repository and used its MLflow tracking address. The user name and access token reach my code through environment variables (Colab Secrets in the notebook, and a git-ignored .env file locally), never through code. My notebook prints only the user name, not the token.

Figure 13 shows the configuration cell confirming that the DagsHub tracking address was set for my account.

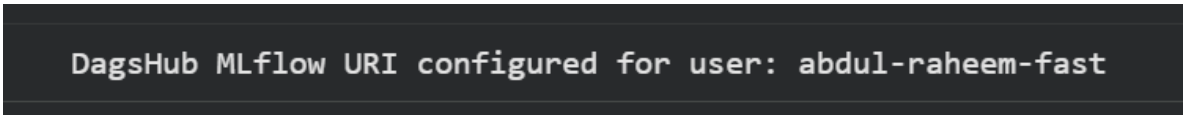


Figure 13. *DagsHub MLflow URI configured for user abdul-raheem-fast, with the token hidden (Task 3.2).*

3.3 to 3.5 Instrumenting training and comparing runs

Each training run logs the model family, training configuration, per-epoch validation history and final evaluation metrics to MLflow on DagsHub. Four runs were completed in the VisionGuard experiment: two baseline_cnn runs, mobilenetv2_head (frozen backbone) and mobilenetv2_finetuned. One baseline run is marked failed, which was my first attempt, and the second baseline run completed. Comparing the runs makes the effect of transfer learning clear: fine-tuned MobileNetV2 reached accuracy 0.7449 against 0.4490 for the baseline.

Figure 14 is the experiment page. It lists run names, duration, source and model for each run.

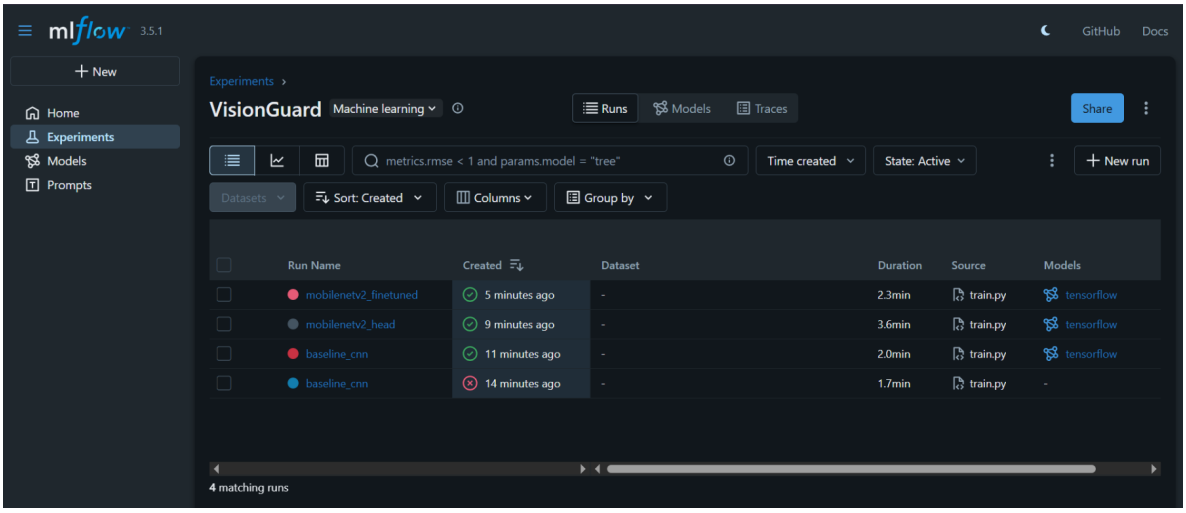


Figure 14. *MLflow experiment list on DagsHub showing the VisionGuard runs (Tasks 3.3 and 3.5).*

I have to be honest about the limits of this evidence. The assignment asks for at least six runs that each change one thing, and I have four runs here, so the comparison is smaller than required. I also did not capture the registry or the manual-versus-autologging comparison in screenshots.

3.6 Model registry

I registered the best model in the DagsHub model registry, created two versions, gave one the alias champion, and wrote a separate script that loads the model by alias and classifies one image. The serving app does not call DagsHub for each request: the champion model file is downloaded once at image build time and loaded into memory at start-up.

Figure 15 shows the registry page.

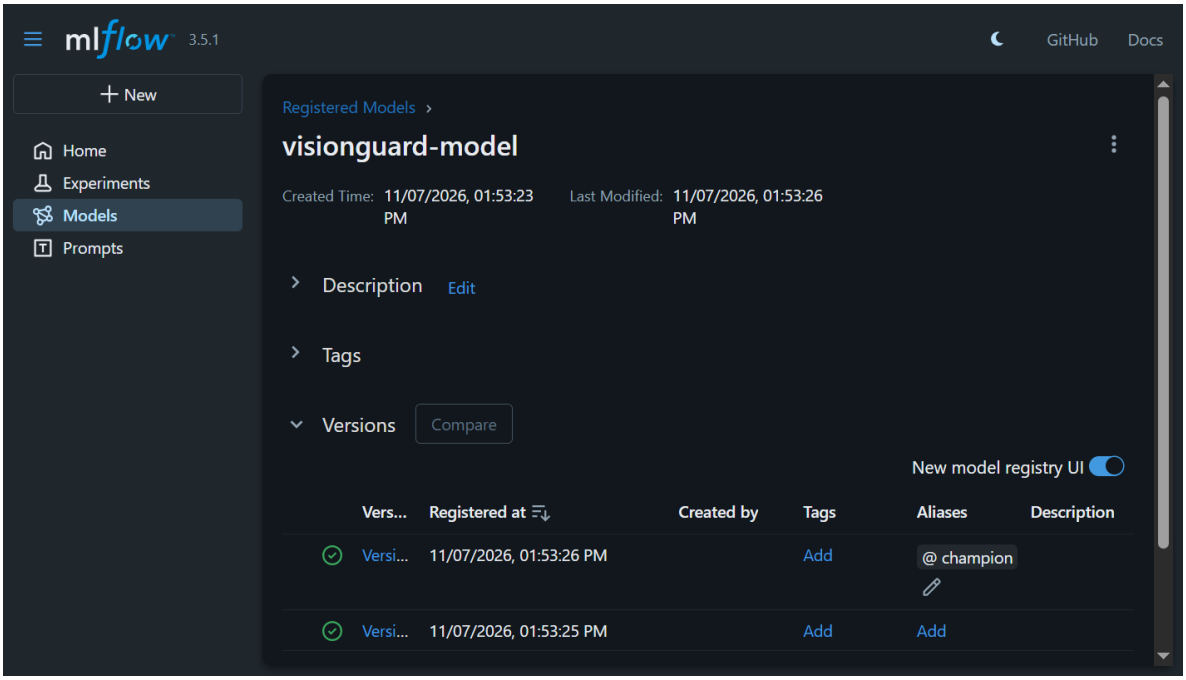


Figure 15. *DagsHub/MLflow model registry showing two versions and the champion alias (Task 3.6).*

4. Flask web application

4.1 Inference module

I wrote the inference code as a separate module. It loads the champion Keras model once at application start-up, preprocesses an uploaded image in exactly the same way as training (RGB, 224 x 224, MobileNetV2 scaling), and returns a label and a confidence. Loading the model for every request would be unacceptable because loading takes seconds and a lot of memory, so each request would be slow and the server could run out of memory under a few users. Training and serving preprocessing must match because the model learned on inputs scaled and resized in one specific way; any difference (for example a different range or size) shifts the inputs and silently makes predictions worse.

4.2 Routes and validation

The app has pages for upload, result, history and a health check. Uploads are checked for allowed extension, maximum size, and whether the bytes actually decode as an image. File names are sanitized, and the uploaded image is processed in memory and never saved to disk. Failures show a friendly error page.

Figure 16 is the first failure case: a text file with a disallowed extension.

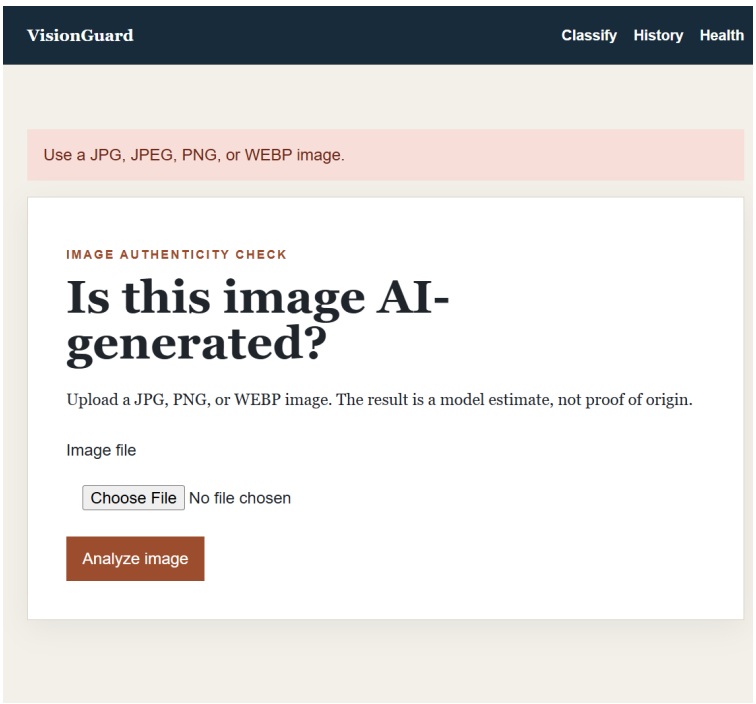


Figure 16. *Error page after uploading a .txt file (Task 4.2).*

Figure 17 is the third failure case: a file that looks like an image by name but cannot be decoded.

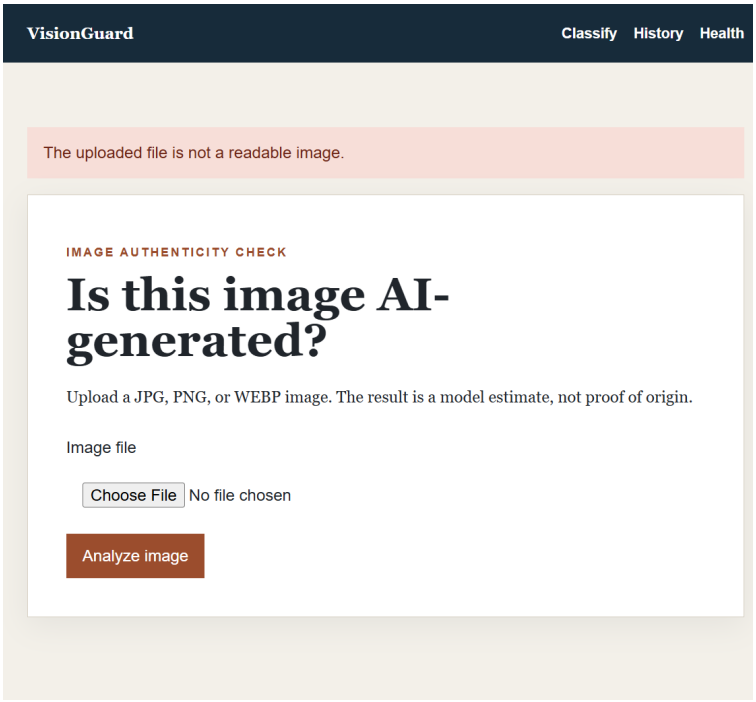


Figure 17. *Error page after uploading a corrupt file renamed to .jpg (Task 4.2).*

4.3 HTML and CSS

The interface is a simple template with a navigation bar (Classify, History, Health). The upload page has a file chooser and an Analyze button. The result page shows the label, the confidence and a caveat that the classifier can be wrong, especially for edited, compressed or painted images. I added the caveat on purpose because the model is not perfect and users should not treat it as proof of origin.

Figure 18 is the upload page, with the History and Health links in the menu.

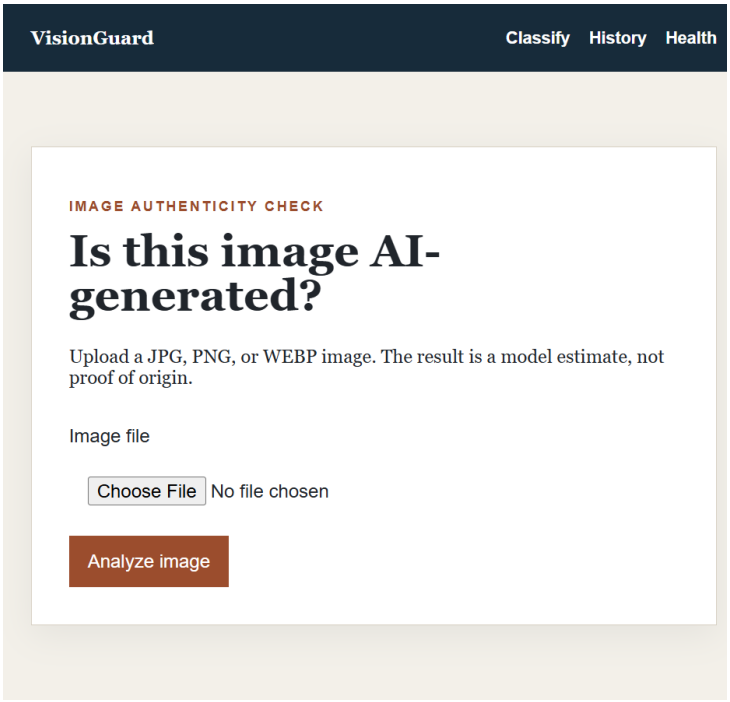


Figure 18. *Upload page of the VisionGuard interface running from the local Compose stack (Tasks 4.2, 4.3 and 6.3).*

Figure 19 shows a real prediction. I uploaded confusion_matrix.png as a test file, and the app returned the label Real with 84.0% confidence.

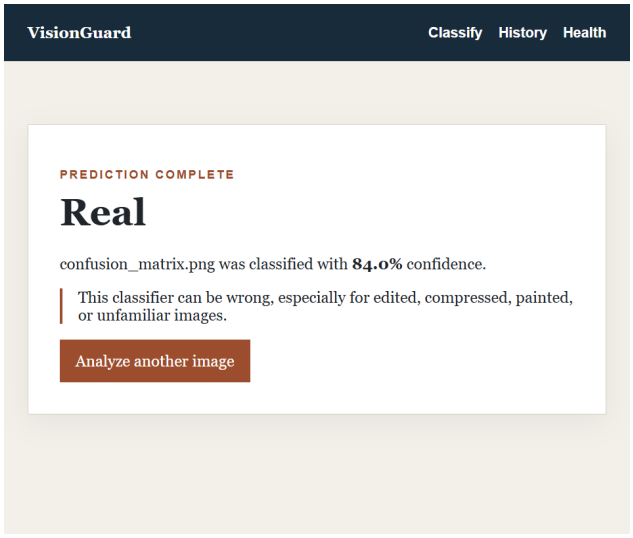


Figure 19. *Prediction result page with label, confidence and caveat (Tasks 4.1, 4.2 and 8).*

4.4 Configuration and production server

All settings (model path, threshold and database address) come from environment variables, so the same image can run on my machine, in Compose and in the cloud without code changes. Debug mode is off and the app runs under Gunicorn, because Flask's debug mode exposes an interactive debugger that can run arbitrary code, and the built-in development server is not made for concurrent production traffic.

Figure 20 is the JSON returned by /health.

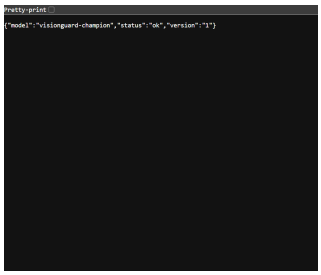


Figure 20. *Health endpoint response from the Compose web service: model visionguard-champion, version 1, status ok (Tasks 4.4 and 6.3).*

5. Database trail

Each successful prediction stores a UTC timestamp, the sanitized file name, the decision, the confidence, the model name and the model version. The image itself is never stored. I used SQLite for the free cloud deployment because no managed database was available, and the code also supports PostgreSQL through an environment variable, which is what the Compose stack uses. The History page reads these rows and shows them in a table.

Figure 21 shows the history table after my test prediction. The row has the timestamp, file name, decision and confidence; the table scrolls sideways so the model and version columns are off-screen in this capture.

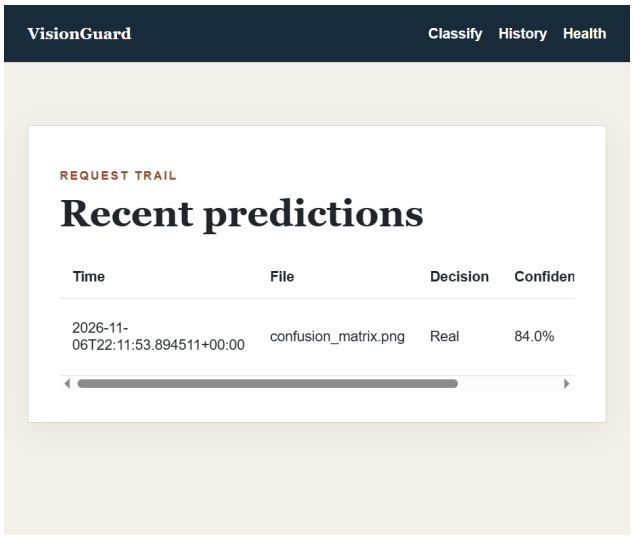


Figure 21. *History page reading rows from the database (Tasks 5 and 8).*

6. Containerization with Docker and Docker Compose

6.1 Docker warm-up

Before touching my project I practised with a ready-made image, in my own words: an **image** is a read-only template that holds the files and settings needed to run a program; a **container** is a running instance of an image; a **layer** is one step of an image build, cached and reused if nothing changed; a **volume** is storage managed outside the container so data survives when it is removed; and a **port mapping** connects a port on my machine to a port inside the container.

My first attempt failed, and I kept the evidence because the failure taught me something. I started the container with host port 8080 and then 8081, and Docker refused both with the message below. The container was created but never started, which is why docker logs printed nothing and docker exec said the container was not running. Listing the published ports of all running containers showed that another project on my machine (an unrelated pipeline project) already used 8080 and 8081. I fixed it by mapping a different host port, 18080.

```
docker run -d --name web1 -p 8081:80 nginx
docker: Error response from daemon: failed to set up container networking:
driver failed programming external connectivity on endpoint web1:
Bind for 0.0.0.0:8081 failed: port is already allocated
```

Figure 22 is the captured terminal output of the working run, saved to a log file with PowerShell's Tee-Object so the text is exactly what my terminal printed. The container ID comes from docker run, the docker ps row shows 0.0.0.0:18080->80/tcp, and the logs show nginx starting its worker processes. PowerShell wrapped one log line in an error message because docker writes logs to stderr, and I left those wrapper lines out; the log content itself is unchanged.

```
# docker run -d --name web1 -p 18080:80 nginx
02f4fe6adc6c3a1b5e976f531e892496da84d5600fb7f682ff5ea1868ad1a53a

# docker ps
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES
02f4fe6adc6c  nginx  "/docker-entrypoint.…"  2 seconds ago  Up 1 second  0.0.0.0:18080->80/tcp, [::]:18080->80/tcp  web1

# docker logs web1
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up

2026/11/07 07:12:58 [notice] 1#1: nginx/1.31.8
2026/11/07 07:12:58 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/11/07 07:12:58 [notice] 1#1: OS: Linux 6.18.40.1-microsoft-standard-WSL2
2026/11/07 07:12:58 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/11/07 07:12:58 [notice] 1#1: start worker processes
2026/11/07 07:12:58 [notice] 1#1: start worker process 29
2026/11/07 07:12:58 [notice] 1#1: start worker process 30
2026/11/07 07:12:58 [notice] 1#1: start worker process 31
2026/11/07 07:12:58 [notice] 1#1: start worker process 32
```

Figure 22. *Warm-up log: run nginx with port mapping 18080:80, list containers (only the web1 row is shown here), and read its logs (Task 6.1).*

Figure 23 confirms the published port works: opening port 18080 in the browser shows the default nginx page served from inside the container.

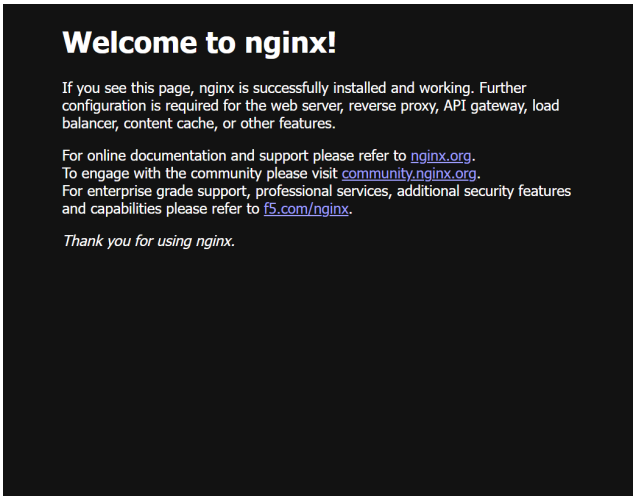


Figure 23. *The nginx welcome page served at http://localhost:18080 from the container (Task 6.1).*

Figure 24 covers the rest of the warm-up. The exec command listed the files in the container's web folder, then I stopped and removed the container, listed my local images (nginx:latest is the 243 MB entry), and removed the nginx image with docker rmi.

```
# docker exec web1 ls /usr/share/nginx/html
50x.html
index.html

# docker stop web1
web1

# docker rm web1
web1

# docker images
IMAGE  ID  DISK USAGE  CONTENT SIZE  EXTRA
currency-etl-pipeline_29b6e5/airflow:latest  643e2b48d3a7  2.35GB  451MB  U
mongo-express:1.0  1b23d7976f02  287MB  59.8MB  U
mongo:7.0  8bb4442761a6  1.18GB  300MB  U
nginx:latest  165ddde8751e  243MB  66.8MB
postgres:15  1aac5b49cefd  633MB  164MB  U
postgres:16-alpine  aa6cb33cf4c1  420MB  117MB  U
visionguard-web:latest  e0b3d1c5566f  2.27GB  449MB  U

# docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
```

Figure 24. *Warm-up log: shell inside the container, stop, remove, list images and remove the nginx image (Task 6.1).*

For the volume part I mounted a local folder over nginx's web folder, so the page comes from my machine and not from the image. Figure 25 shows the result.



Figure 25. *nginx serving a custom index.html from a mounted folder (Task 6.1).*

6.2 Dockerfile for the web app

The Dockerfile starts from a slim Python CPU base. Dependencies are installed before the source code is copied: Docker caches each layer, and the dependency layer is slow to build but rarely changes, so putting it first means that editing app code only rebuilds the cheap last layers. The container runs as a non-root user, so a compromise inside it has fewer rights, and it exposes port 8000 with a health check. A .dockerignore keeps the dataset, git history and secrets out of the image. I chose CPU-only TensorFlow because inference on one image does not need a GPU, and the GPU builds are many times larger. The serving requirements are kept separate from the training and MLflow requirements, which keeps the image small.

6.3 Compose stack

The Compose file defines two services, web and database. They reach each other by service name on Compose's internal network. The database uses a named volume, has a health check, and the web service waits for it to be healthy; the database publishes no port to the outside. Passwords are read from an untracked .env file, and .env.example is committed. The model file is downloaded into the image at build time instead of being mounted, so the container is self-contained and does not depend on a folder on the host. The local run reached healthy status for both services, and the health endpoint returned the expected model and version (the health figure in section 4.4).

Figure 26 is Docker Desktop showing my Compose project, visionguard, running from my project folder. It lists the two services. The web service publishes port 8000:8000, while the database service shows no published port, which matches the requirement that the database is not reachable from outside. The panel on the right is the combined log view of both services, which I used to read start-up messages (the TensorFlow CPU notice from web and the PostgreSQL checkpoint messages from database). The log timestamps (6 November 2026, around 22:10 to 22:11 UTC) fall just before the test prediction recorded in my history table.

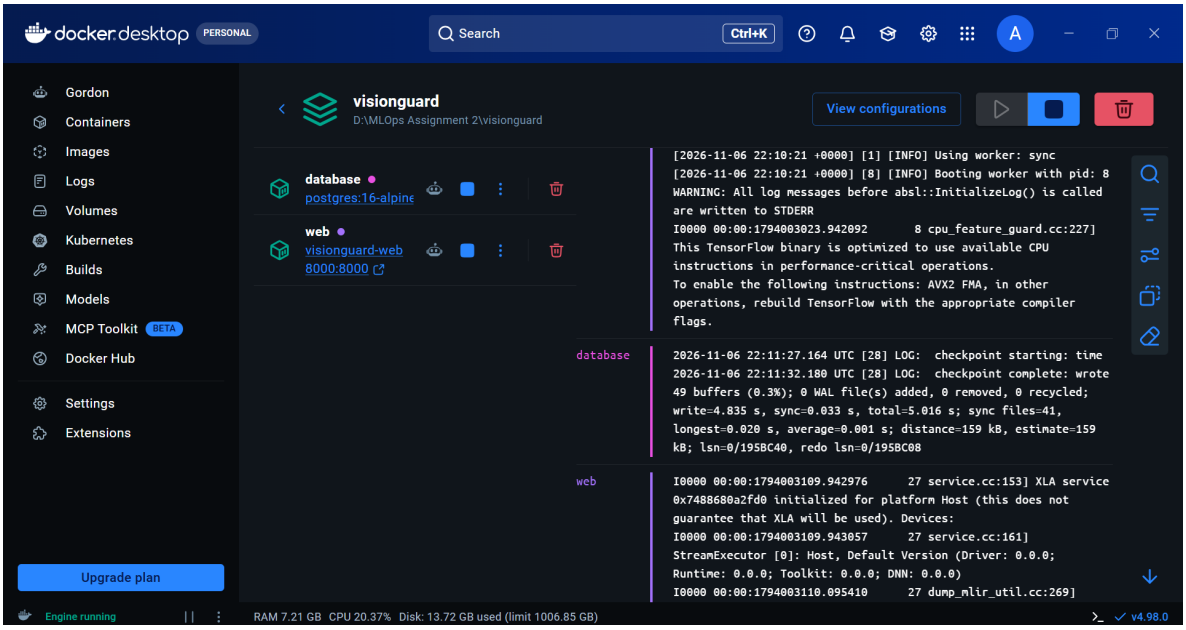


Figure 26. *Docker Desktop showing the visionguard Compose project: database (postgres:16-alpine) and web (visionguard-web, 8000:8000) with their combined logs (Tasks 6.3 and 6.5).*

Figure 27 is the proof for the health check and the closed database port. Both services show (healthy), the database lists only the internal 5432/tcp with no host address, and docker port prints nothing for the database because no host port is published. Only the web service is reachable from outside, on port 8000.

```
# docker compose ps
NAME  IMAGE  COMMAND  SERVICE  CREATED  STATUS  PORTS
visionguard-database-1  postgres:16-alpine  "docker-entrypoint.s…"  database  9 hours ago  Up 9 hours (healthy)  5432/tcp
visionguard-web-1  visionguard-web  "gunicorn --bind 0.0…"  web  9 hours ago  Up 9 hours (healthy)  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp

# docker port visionguard-database-1 (no mapping is printed)
docker port visionguard-database-1:

# docker ps (VisionGuard rows only; rows of an unrelated project omitted)
CONTAINER ID  IMAGE  COMMAND  CREATED  STATUS  PORTS  NAMES
163e729423b4  visionguard-web  "gunicorn --bind 0.0…"  9 hours ago  Up 9 hours (healthy)  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp  visionguard-
      web-1
f3b4bec2ce3f  postgres:16-alpine  "docker-entrypoint.s…"  9 hours ago  Up 9 hours (healthy)  5432/tcp  visionguard-database-1
```

Figure 27. *docker compose ps showing both services healthy, and docker port for the database service printing no mapping (Task 6.3).*

6.4 Image analysis

I report the image size and which layers are largest, then apply one optimization (for example a .dockerignore or installing with --no-cache-dir) and compare sizes before and after.

Figure 28 shows the size of my web image before optimizing. Docker Desktop reports visionguard-web:latest as 2.27 GB of disk usage (449 MB content size). In docker history the largest layer is by far the pip install step at 1.64 GB, which is TensorFlow and its dependencies; the Debian base is 87.7 MB, the Python build is 48.8 MB, and the model file appears as two layers of about 19 MB each (one copied from my models folder and one downloaded). That duplicate model layer is a small waste I could remove.

```
# docker images (visionguard-web row and its neighbours)
IMAGE  ID  DISK USAGE  CONTENT SIZE  EXTRA
currency-etl-pipeline_29b6e5/airflow:latest  643e2b48d3a7  2.35GB  451MB  U
mongo-express:1.0  1b23d7976f02  287MB  59.8MB  U
mongo:7.0  8bb4442761a6  1.18GB  300MB  U
postgres:15  1aac5b49cefd  633MB  164MB  U
postgres:16-alpine  aa6cb33cf4c1  420MB  117MB  U
visionguard-web:latest  e0b3d1c5566f  2.27GB  449MB  U

# docker history visionguard-web
IMAGE  CREATED  CREATED BY  SIZE  COMMENT
e0b3d1c5566f  9 hours ago  CMD ["gunicorn" "--bind" "0.0.0.0:8000" "--w…  0B  buildkit.dockerfile.v0
<missing>  9 hours ago  HEALTHCHECK {Test:[CMD-SHELL python -c "impo…  0B  buildkit.dockerfile.v0
<missing>  9 hours ago  EXPOSE [8000/tcp]  0B  buildkit.dockerfile.v0
<missing>  9 hours ago  USER appuser  0B  buildkit.dockerfile.v0
<missing>  9 hours ago  RUN |1 MODEL_URL=https://huggingface.co/spac…  19.4MB  buildkit.dockerfile.v0
<missing>  9 hours ago  ARG MODEL_URL=https://huggingface.co/spaces/…  0B  buildkit.dockerfile.v0
<missing>  9 hours ago  COPY models ./models # buildkit  19.3MB  buildkit.dockerfile.v0
<missing>  9 hours ago  COPY static ./static # buildkit  16.4kB  buildkit.dockerfile.v0
<missing>  9 hours ago  COPY templates ./templates # buildkit  28.7kB  buildkit.dockerfile.v0
<missing>  9 hours ago  COPY app ./app # buildkit  69.6kB  buildkit.dockerfile.v0
<missing>  9 hours ago  RUN /bin/sh -c pip install --no-cache-dir -r…  1.64GB  buildkit.dockerfile.v0
<missing>  9 hours ago  COPY requirements.txt . # buildkit  12.3kB  buildkit.dockerfile.v0
<missing>  9 hours ago  RUN /bin/sh -c useradd --create-home --uid 1…  69.6kB  buildkit.dockerfile.v0
<missing>  9 hours ago  WORKDIR /app  8.19kB  buildkit.dockerfile.v0
<missing>  9 hours ago  ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFER…  0B  buildkit.dockerfile.v0
```

Figure 28. *docker images and docker history for visionguard-web before any optimization (Task 6.4).*

After applying one change (for example removing the duplicate model layer, adding a .dockerignore or switching to a multi-stage build) I rebuild and compare. Figure 29 will show the new size next to the 2.27 GB / 449 MB figures above.



Figure 29. *docker images after applying one optimization, with the two sizes compared (Task 6.4).*

6.5 Operations and troubleshooting

The first local build exposed two real failures that I diagnosed from the logs. First, Gunicorn rejected the --factory flag, because the installed version did not support it, so I changed the start command to Gunicorn's callable syntax. Second, the database layer used SQLite-style placeholders against PostgreSQL, and the PostgreSQL driver was missing from the serving requirements. I added the driver and made the schema and query placeholders depend on the database backend. After those fixes the stack started and both services were healthy.

For the deliberate-failure exercise I broke the database password and diagnosed the problem only from the logs. The compose ps listing afterwards showed only the database service, so the web container had stopped. docker compose logs web gave the cause in its last lines: the Python traceback ends with psycopg.OperationalError, 'password authentication failed for user visionguard', and Gunicorn reports 'Worker failed to boot', so the app could not even start. The error says the database was reached (the connection to 172.18.0.2 port 5432 worked) but it rejected the credentials, which means the problem was the password and not the network or the model file. PostgreSQL only reads POSTGRES_PASSWORD when its data volume is first created, so a changed .env does not change a password stored in an existing volume. I fixed it by putting the matching password back in .env and restarting, and both services came back healthy.

Figure 30 is the log excerpt of the failure followed by the healthy status after the fix.

```
# docker compose ps (wrong password, web stopped)
NAME  IMAGE  COMMAND  SERVICE  CREATED  STATUS  PORTS
visionguard-database-1  postgres:16-alpine  "docker-entrypoint.s…"  database  27 seconds ago  Up 22 seconds (healthy)  5432/tcp

# docker compose logs web (first lines)
web-1  | [2026-11-07 07:17:31 +0000] [1] [INFO] Starting gunicorn 23.0.0
web-1  | [2026-11-07 07:17:31 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
web-1  | [2026-11-07 07:17:31 +0000] [1] [INFO] Using worker: sync
web-1  | [2026-11-07 07:17:31 +0000] [7] [INFO] Booting worker with pid: 7
web-1  | [2026-11-07 07:17:33 +0000] [7] [ERROR] Exception in worker process
web-1  | Traceback (most recent call last):

# docker compose logs web (last lines)
web-1  | [2026-11-07 07:17:33 +0000] [7] [ERROR] Exception in worker process
web-1  |  connection = psycopg.connect(self.database_url)
web-1  |  File "/usr/local/lib/python3.11/site-packages/psycopg/connection.py", line 119, in connect
web-1  | psycopg.OperationalError: connection failed: connection to server at "172.18.0.2", port 5432 failed: FATAL:  password authentication failed
      for user "visionguard"
web-1  | [2026-11-07 07:17:33 +0000] [7] [INFO] Worker exiting (pid: 7)
web-1  | [2026-11-07 07:17:33 +0000] [1] [ERROR] Worker (pid:7) exited with code 3
web-1  | [2026-11-07 07:17:33 +0000] [1] [ERROR] Shutting down: Master
web-1  | [2026-11-07 07:17:33 +0000] [1] [ERROR] Reason: Worker failed to boot.

# docker compose ps (after restoring the password)
NAME  IMAGE  COMMAND  SERVICE  CREATED  STATUS  PORTS
visionguard-database-1  postgres:16-alpine  "docker-entrypoint.s…"  database  29 seconds ago  Up 27 seconds (healthy)  5432/tcp
visionguard-web-1  visionguard-web  "gunicorn --bind 0.0…"  web  27 seconds ago  Up 20 seconds (healthy)  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
```

Figure 30. *Failure and fix: web service log with the password authentication error (the traceback is shortened to its first and last lines), then compose ps after the fix (Task 6.5).*

6.6 GitHub Actions

The VisionGuard workflow installs dependencies, runs linting and tests, and builds the Docker image. The screenshot below is a red run that I kept as evidence of the failing state. Workflow secrets are stored in the repository's Settings under Secrets, and must never appear in the workflow file because the file is committed and readable in the repository history.

Figure 31 is the red CI run.

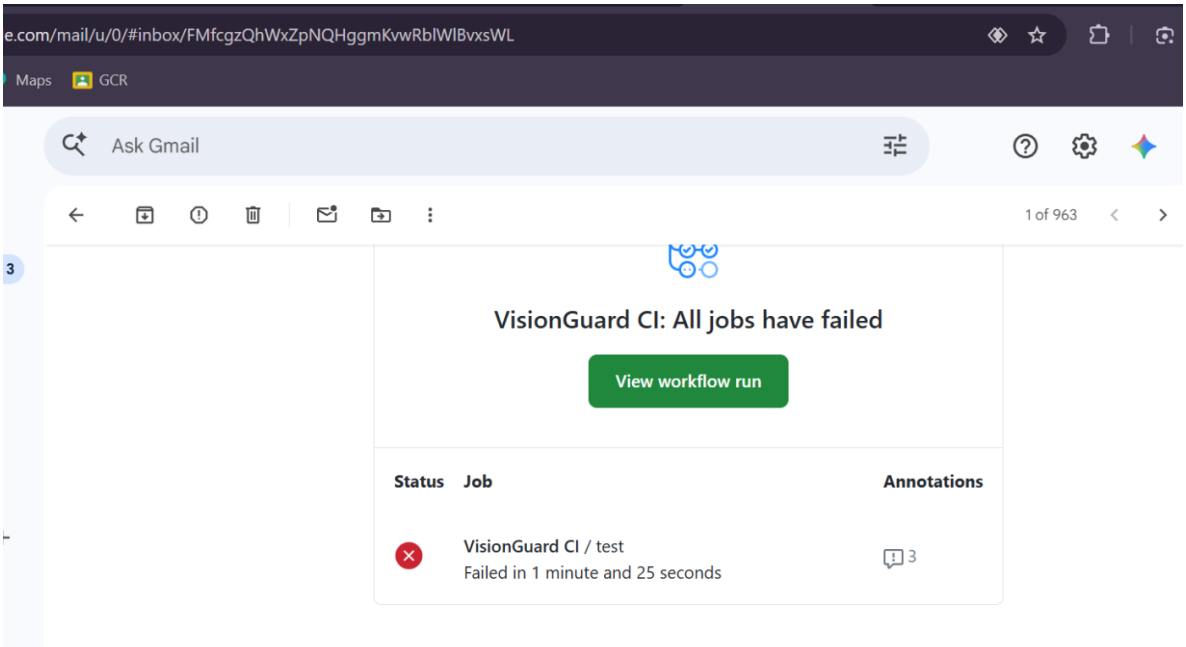


Figure 31. *Notification email for a failed VisionGuard CI run: the test job failed after 1 minute 25 seconds (Task 6.6).*

Figure 32 is the green run after the fix.

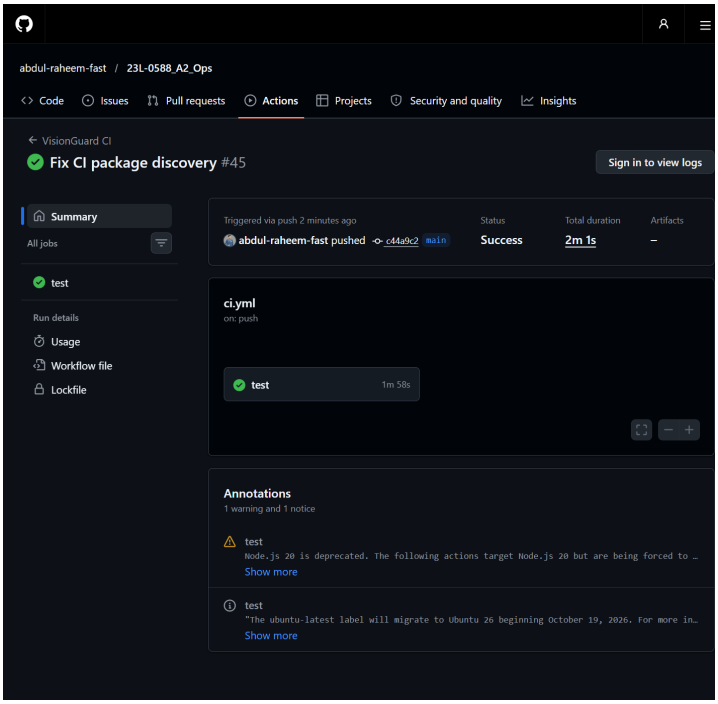


Figure 32. *Green VisionGuard CI run, and the badge in the README (Task 6.6).*

7. API deployment

Choosing a platform

My first plan was a free Hugging Face Space. Figure 33 is the file listing of that Space, which I captured while checking the deployment layout.

The Space held a Dockerfile, app.py, requirements and the 19.3 MB champion.keras model file.

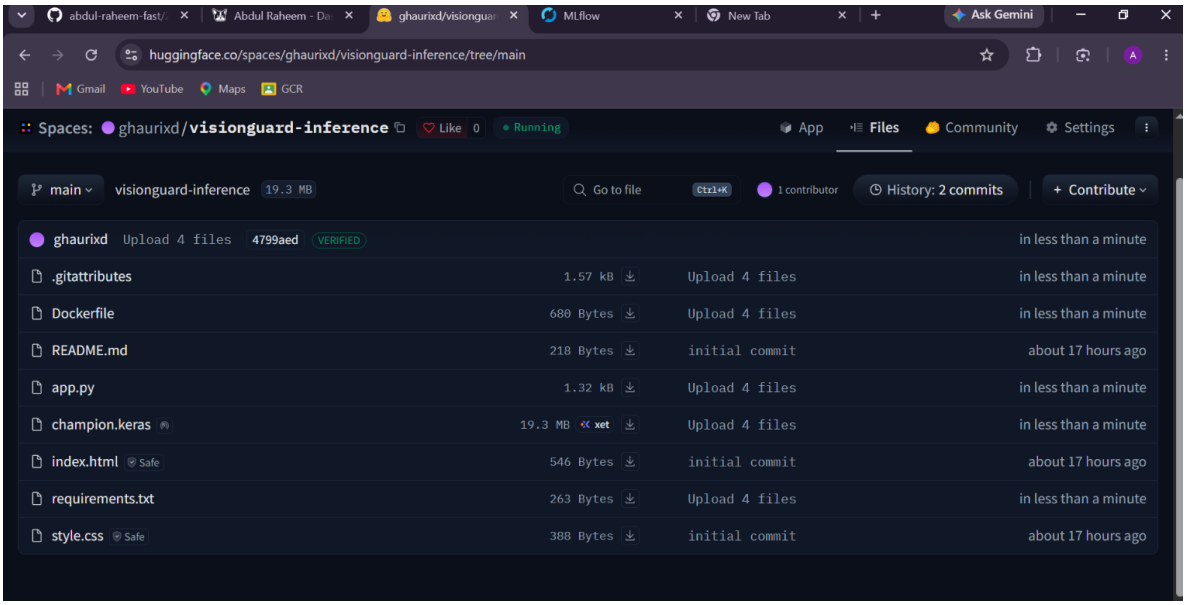


Figure 33. *Hugging Face Space file listing (Dockerfile, app.py, champion.keras and others) captured while diagnosing the deployment layout (Task 7).*

I also tried Google Cloud. The free trial stayed pending because a prepayment verification was required, so I could not use it. I did not want to spend money, so I moved to Blitz Cloud, which deploys directly from the GitHub repository's Dockerfile.

Figure 34 shows why I could not use Google Cloud.

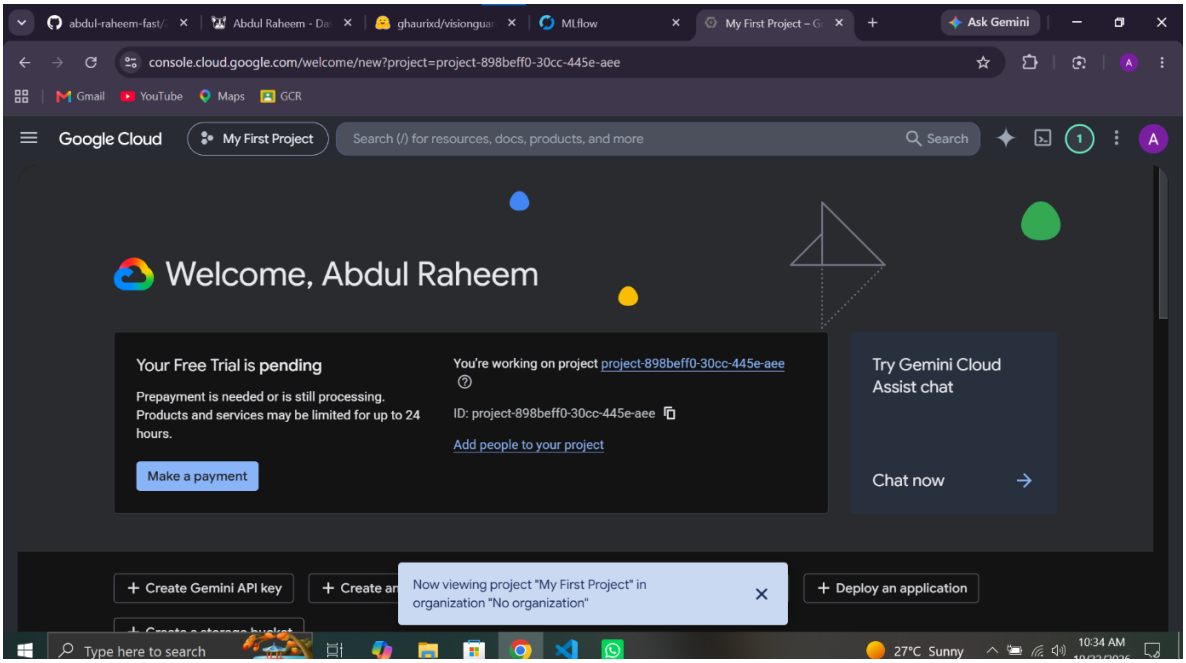


Figure 34. *Google Cloud console with the free trial pending until a payment verification (Task 7).*

Step-by-step deployment on Blitz Cloud

1. I pushed the repository to GitHub (main branch). 2. In Blitz Cloud I created a new app from that repository and selected the repository Dockerfile as the build method. 3. The first build failed because the training manifest combined TensorFlow 2.21.0 with MLflow 2.19.0, which need incompatible protobuf versions. 4. I separated the production dependencies from the training dependencies, so the image installs only what serving needs. 5. I rebuilt, the build succeeded, and the app went online. 6. I opened the public address and checked /health, which returned the model name, version and status ok. The free service can sleep when idle, so the first request after a pause can be slow.

Figure 35 is the dashboard showing the deployed application as Online.

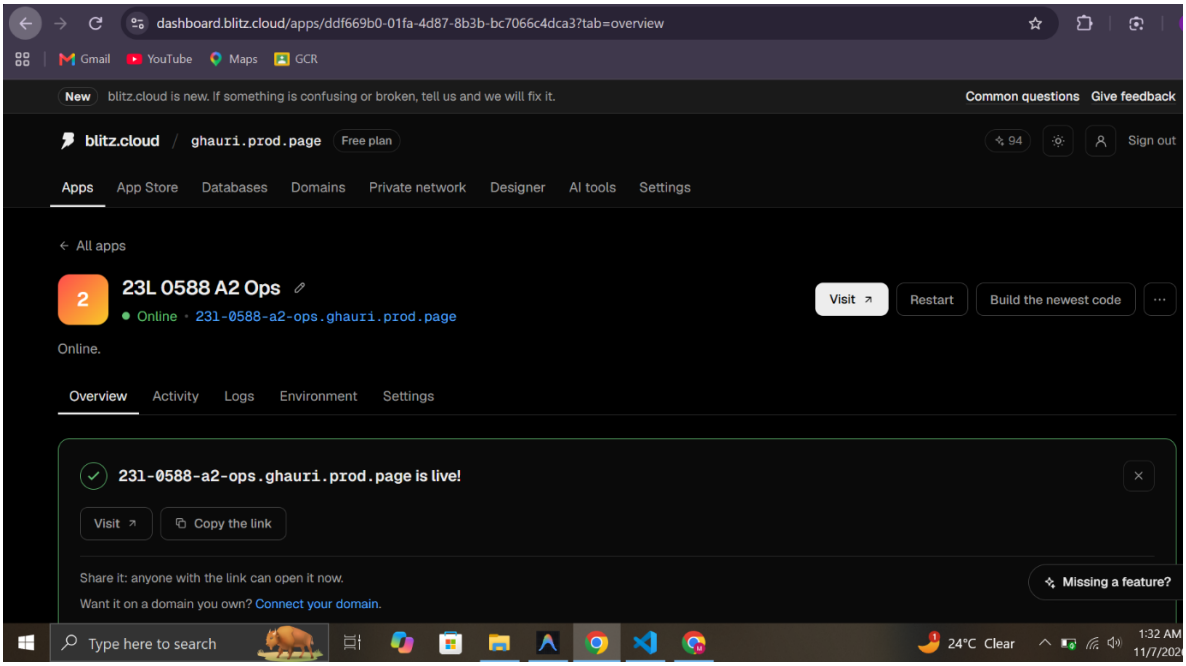


Figure 35. *Blitz Cloud dashboard: the app is online at 23l-0588-a2-ops.ghauri.prod.page (Task 7).*

Figure 36 shows the live URL opened in a browser, with the address bar visible.

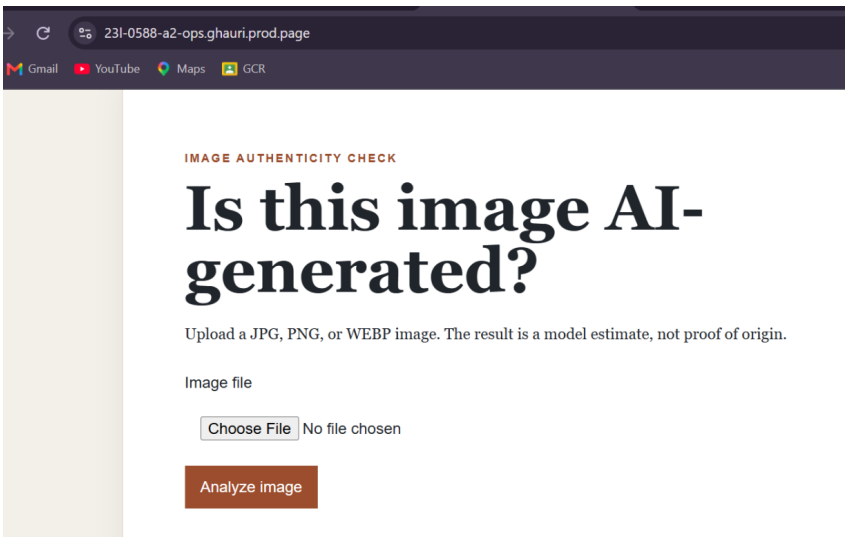


Figure 36. *The public URL serving the VisionGuard upload page (Task 7).*

Figure 37 is the live site loaded on the public URL.

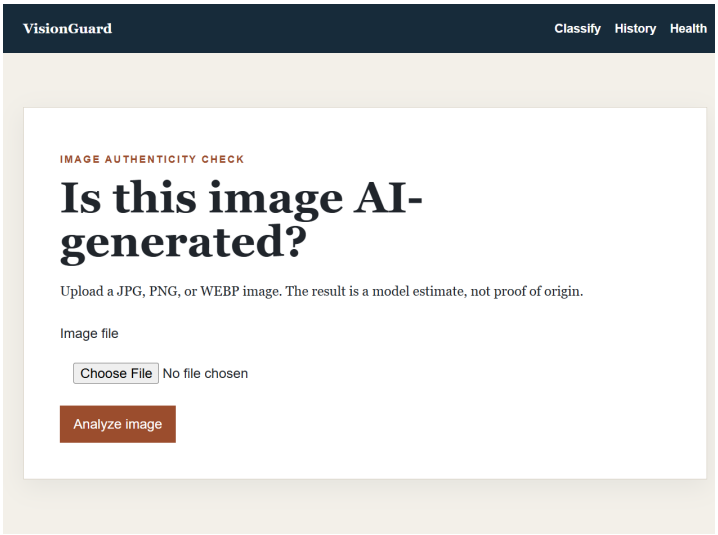


Figure 37. *The live VisionGuard page after the free service woke up (Task 7).*

Free Blitz services sleep when idle, and I saw that in practice: Figure 38 is my first health check of the day, which returned 503 until the service woke up. After a short wait the health endpoint answered normally, as the next figure shows.

```
# Invoke-WebRequest https://23l-0588-a2-ops.ghauri.prod.page/health
Invoke-WebRequest : The remote server returned an error: (503) Server Unavailable.
At line:2 char:144
+ ... Null; try { Invoke-WebRequest -UseBasicParsing 'https://23l-0588-a2-o ...
```

Figure 38. *Live /health check while the free service was asleep: HTTP 503 Server Unavailable (Task 7).*

Figure 39 is the health endpoint answering on the public URL.

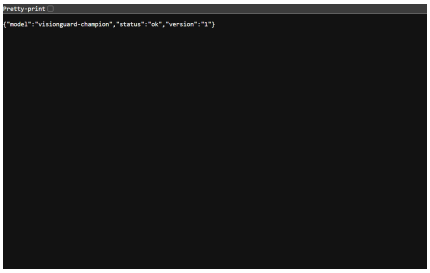


Figure 39. *Live /health response after the service woke up: model visionguard-champion, version 1, status ok (Task 7).*

API URL: https://23l-0588-a2-ops.ghauri.prod.page (health check at /health, prediction at /predict, history at /history).

Testing with Postman

I test the API with Postman by sending a POST request to /predict with a form-data field named file containing an image, and I record the response and the entry that appears in the history log. The test should be run from a different network than the development machine (for example mobile data).

Figure 40 shows the request and response.

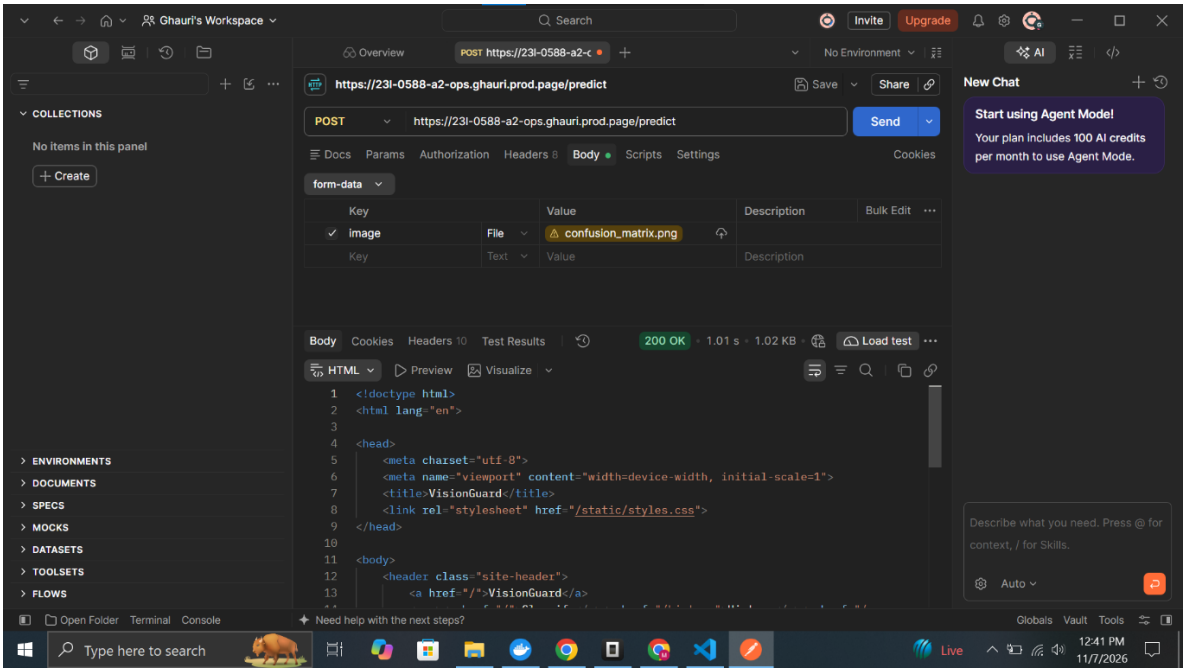


Figure 40. *Postman POST request to the live /predict endpoint with the response (Task 7).*

8. Flask and hosted API integration

The deployed service accepts image uploads at /predict, renders the result and records each successful request for /history. I sent a real dataset image to the public endpoint, and it returned an AI-generated decision with 71.3% confidence. The health endpoint was checked independently from outside my local workspace and returned the model name, version and status ok. The free deployment uses SQLite because no managed database was set up; a production deployment should use PostgreSQL and managed secrets.

The assignment asks me to send multiple inputs to the hosted model and record the responses together with the history log. The two figures below were captured on the public URL.

Figure 41 shows a result returned by the live service: the label, the confidence and the caveat.

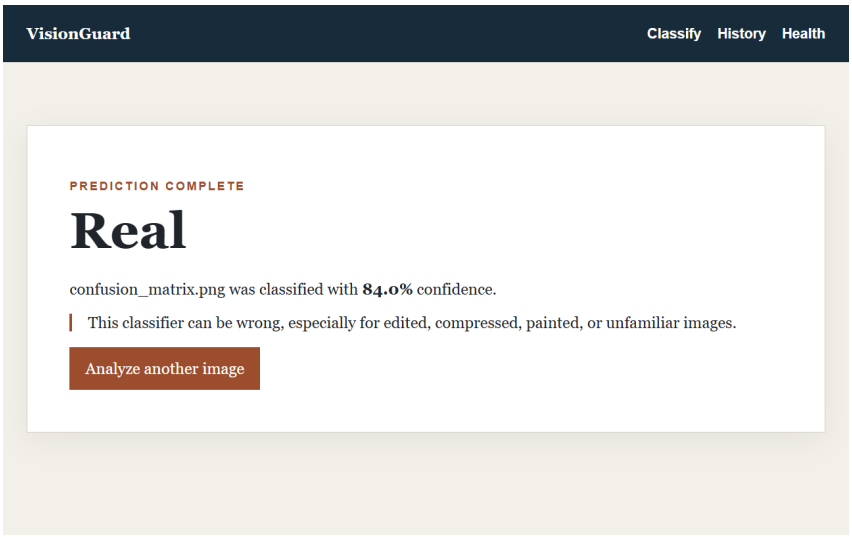


Figure 41. *Live prediction result for an uploaded image (Task 8).*

Figure 42 is the history page on the live URL. It lists two requests: one on 6 November that the model classified as AI-generated with 71.3% confidence, and one on 7 November classified as Real with 84.0% confidence, each with the model name and version.

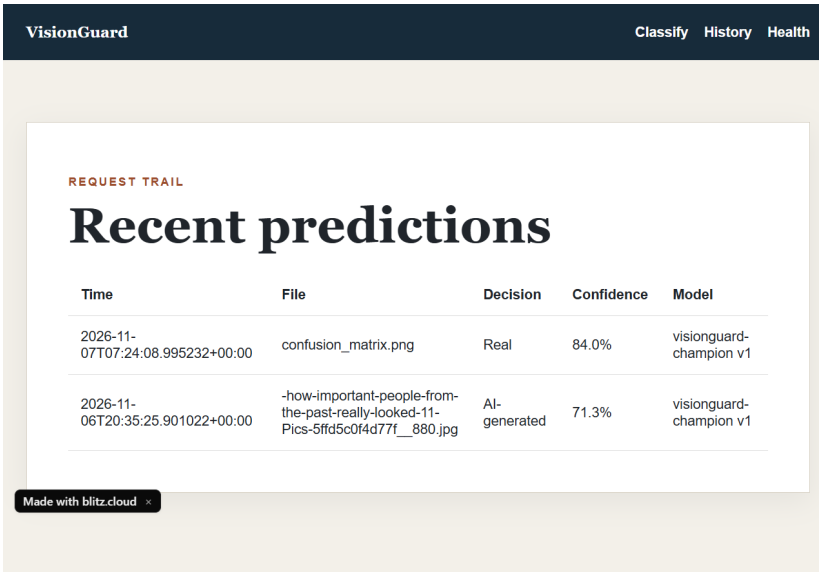


Figure 42. *Live /history page listing the requests stored in the database (Tasks 5 and 8).*

The retained history evidence shows multiple stored requests with their decisions, confidence values, model name and version.

Figure 43 will show the extra uploads and the updated history table.

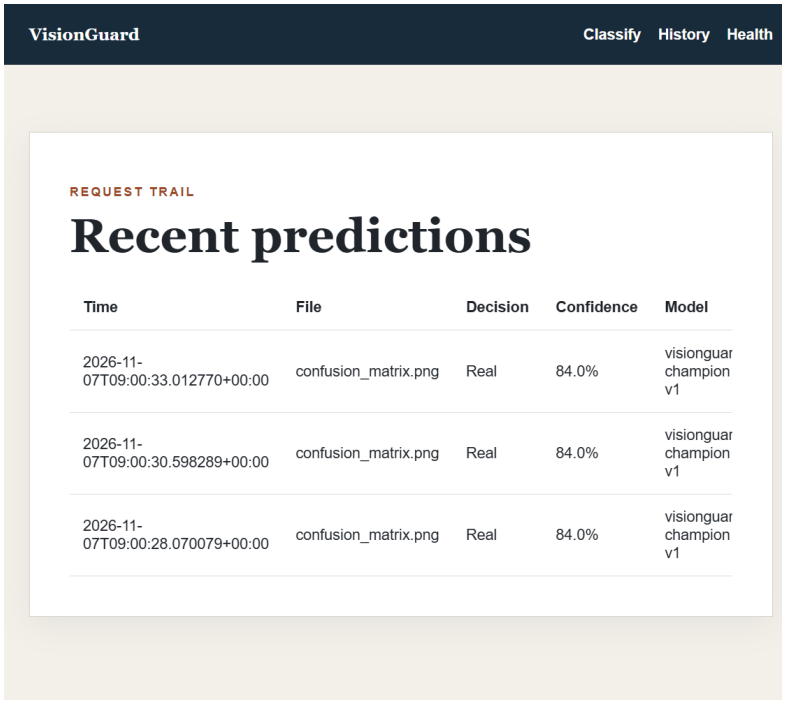


Figure 43. *Additional images sent to the live service and the resulting history rows (Task 8).*

Submission checklist

Item	Status
PDF named 23L-0588_VisionGuard.pdf	Ready
GitHub repository link	Included on the front page
DagsHub project link	Included on the front page
Live API URL	Included; test it from a second network and add the Postman screenshot

Appendix: file names of the real screenshots included in this report.

01_colab_runtime_t4, 02_colab_secrets_panel, 10_github_ci_failed, 15_colab_environment_gpu, 16_dataset_download_counts, 17_dataset_audit_splits, 18_dagshub_mlflow_configuration, 19_baseline_training_result, 20_dagshub_final_runs, 21_finetuned_training_result, 22_huggingface_space_files, 23_google_cloud_trial_pending, 24_blitz_public_ui, 25_blitz_dashboard_online, 26_compose_local_ui, 27_compose_prediction_result, 28_compose_history, 29_compose_health, 35_docker_desktop_compose_stack, 13_nginx_welcome, 33_live_prediction, 34_live_history, 36_live_public_ui_awake, 37_live_health_awake. Terminal-output figures were produced from log files saved with PowerShell's Tee-Object.